**LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY**
**(UGC Autonomous)**
Approved by AICTE | Affiliated to Osmania University | Estd.2003
Accredited by **NBA**, Accredited '**A**' Grade by NAAC
**Department of Computer Science and Engineering (Data Science)**

# Smart Traffic Management Using YOLO-Based Vehicle Detection

## 1. Abstract

**Smart Traffic Management Using YOLO-Based Vehicle Detection** is an AI-powered traffic monitoring and management system designed to automatically detect, classify, and analyze vehicles from live CCTV/video footage. The system uses the **YOLO (You Only Look Once)** object-detection model to identify vehicles such as cars, buses, trucks, motorcycles, and auto-rickshaws in real time.

The system processes traffic-camera video streams and determines vehicle counts, traffic density, lane occupancy, and congestion levels. Based on the detected traffic conditions, it can provide recommendations for **adaptive traffic-signal timing**, giving longer green-light durations to heavily congested lanes and reducing unnecessary waiting time on low-traffic roads.

The system can also maintain historical traffic statistics, generate congestion reports, and provide a dashboard for traffic authorities to monitor multiple intersections. Advanced versions can incorporate **vehicle tracking, number-plate recognition, accident detection, emergency-vehicle prioritization, and traffic prediction**.

The project demonstrates the practical application of **computer vision, deep learning, object detection, and intelligent decision-making** for developing modern smart-city traffic-management solutions.

## 2. Major Modules

| Module | Description | Key Functions |
|---|---|---|
| **1. User/Admin Management** | Controls access to the traffic-management system | Login, roles, user management |
| **2. Camera Management** | Registers and manages traffic cameras | Camera registration, RTSP/video source configuration, camera status |
| **3. Video Acquisition** | Receives traffic video from cameras or uploaded files | Live stream, video upload, frame extraction |
| **4. YOLO Vehicle Detection** | Detects vehicles from video frames | Car, bus, truck, motorcycle, auto detection |
| **5. Vehicle Classification** | Categorizes detected vehicles | Vehicle type, bounding box, confidence score |
| **6. Vehicle** | Tracks individual vehicles | Vehicle ID, movement direction, |

| Module | Description | Key Functions |
|---|---|---|
| **Tracking** | across consecutive frames | entry/exit tracking |
| **7. Traffic Density Analysis** | Determines traffic conditions for each lane/road | Vehicle count, occupancy, density calculation |
| **8. Lane-wise Traffic Analysis** | Analyzes traffic separately for different lanes | Lane count, queue length, lane congestion |
| **9. Congestion Detection** | Identifies traffic congestion automatically | Low/Medium/High/Severe congestion |
| **10. Adaptive Signal Management** | Recommends or controls signal timing based on traffic | Dynamic green time, red time, priority management |
| **11. Emergency Vehicle Detection** | Detects emergency vehicles when implemented | Ambulance/fire truck/police vehicle identification |
| **12. Traffic Prediction** | Predicts future traffic conditions using historical data | Traffic forecasting, peak-hour prediction |
| **13. Analytics Dashboard** | Displays real-time traffic information | Vehicle count, density, congestion, charts |
| **14. Historical Reports** | Stores and analyzes traffic history | Daily/weekly/monthly reports |
| **15. Alert&Notification** | Generates alerts for abnormal conditions | Heavy congestion, accidents, camera failure |

# 3. Proposed System Flow

```
Traffic CCTV / Video
        ↓
Video Frame Extraction
        ↓
Image Preprocessing
        ↓
YOLO Object Detection
        ↓
Vehicle Classification
        ↓
Vehicle Tracking
        ↓
Lane-wise Vehicle Counting
        ↓
Traffic Density Calculation
        ↓
Congestion Analysis
        ↓
```

```
Adaptive Signal Decision
        ↓
Dashboard + Alerts + Reports
```

## Example

Suppose an intersection has four roads:

| Road | Vehicles | Density | Recommended Action |
|---|---|---|---|
| North | 45 | High | Increase green time |
| South | 12 | Low | Reduce green time |
| East | 68 | Very High | Give priority |
| West | 20 | Medium | Normal timing |

The system can calculate an appropriate signal allocation instead of using a fixed timing such as:

```
North → 60 sec
South → 60 sec
East  → 60 sec
West  → 60 sec
```

Instead, it can dynamically recommend:

```
North → 55 sec
South → 25 sec
East  → 75 sec
West  → 35 sec
```

This is the **smart/adaptive traffic-management component** of the project.

# 4. Technology Stack

| Layer | Technologies |
|---|---|
| **Programming Language** | Python |
| **AI/Deep Learning** | YOLOv8/YOLO11, PyTorch |
| **Computer Vision** | OpenCV |
| **Object Detection** | YOLO |
| **Object Tracking** | ByteTrack / BoT-SORT |
| **Data Processing** | NumPy, Pandas |
| **Backend API** | FastAPI / Flask |
| **Frontend** | React.js / HTML, CSS, JavaScript |

| Layer | Technologies |
|---|---|
| **Database** | PostgreSQL / MySQL |
| **Visualization** | Chart.js / Recharts |
| **Model Training** | Ultralytics YOLO |
| **Annotation** | Roboflow / CVAT / LabelImg |
| **API Testing** | Postman |
| **Version Control** | Git, GitHub |
| **Containerization** | Docker |
| **Deployment** | AWS / Azure / GCP |
| **Monitoring** | Prometheus + Grafana *(advanced)* |

### Recommended Student Stack

For a final-year implementation, I recommend:

**Python + YOLO + OpenCV + FastAPI + React + PostgreSQL + Docker**

This provides enough depth to demonstrate **AI + Computer Vision + Backend + Web Application + Database + Deployment** rather than making the project only a YOLO demonstration.

# 5. Test Data

The project can use both **public datasets and real traffic videos**.

### A. Recommended Public Datasets

| Dataset | Purpose |
|---|---|
| **UA-DETRAC** | Vehicle detection and tracking |
| **BDD100K** | Large-scale driving/road-scene dataset |
| **MIO-TCD** | Vehicle classification |
| **KITTI** | Vehicle/object detection |
| **VisDrone** | Vehicle detection from aerial/drone imagery |
| **COCO** | General object detection and pretrained YOLO evaluation |

For this project, **UA-DETRAC + custom traffic-camera footage** is particularly useful because the system needs vehicle detection and traffic analysis.

### B. Custom Test Video Dataset

Students should additionally create a small custom dataset from traffic videos.

| Test Scenario | Example |
|---|---|
| Low traffic | 5–15 vehicles/frame |
| Medium traffic | 15–30 vehicles/frame |
| Heavy traffic | 30–60 vehicles/frame |
| Traffic jam | 60+ vehicles/frame |
| Daytime | Normal daylight |
| Night | Low-light traffic |
| Rain | Wet-road conditions |
| Multiple lanes | 3–4 lane road |
| Motorcycles | High two-wheeler traffic |
| Mixed traffic | Cars + buses + trucks + motorcycles |
| Occlusion | Vehicles partially blocking each other |
| Emergency vehicle | Ambulance/fire truck |
| Camera variation | Different camera angles |

# 6. Test Dataset Structure

A student can organize the dataset as:

```
traffic_dataset/
│
├── train/
│   ├── images/
│   └── labels/
│
├── validation/
│   ├── images/
│   └── labels/
│
├── test/
│   ├── images/
│   └── labels/
│
└── videos/
    ├── low_traffic.mp4
    ├── medium_traffic.mp4
    ├── heavy_traffic.mp4
    ├── night_traffic.mp4
    └── rainy_traffic.mp4
```

For YOLO training, labels can contain:

```
class_id x_center y_center width height
```

**Example:**

```
2 0.521 0.463 0.214 0.182
```

# 7. Vehicle Classes

A practical project can initially use:

| Class ID | Vehicle |
|---|---|
| 0 | Car |
| 1 | Motorcycle |
| 2 | Bus |
| 3 | Truck |
| 4 | Auto-rickshaw |
| 5 | Van |
| 6 | Emergency Vehicle |

The exact classes depend on the selected dataset and whether the students create a custom dataset.

# 8. Testing&Evaluation

The project should not be evaluated only on whether the YOLO model detects vehicles.

## AI Model Metrics

- **Precision**
- **Recall**
- **mAP@50**
- **mAP@50–95**
- **F1-score**
- **Confidence score**
- **FPS / inference speed**

## Traffic-System Metrics

- Vehicle-count accuracy

- Lane-count accuracy
- Congestion-classification accuracy
- Queue-length estimation accuracy
- Traffic-density estimation error
- Signal-timing decision accuracy
- Average vehicle waiting-time reduction

## Example Final Evaluation

| Metric | Target |
|---|---|
| Vehicle detection precision | ≥ 90% |
| Vehicle detection recall | ≥ 85% |
| mAP@50 | ≥ 90% |
| Vehicle counting accuracy | ≥ 90% |
| Congestion classification | ≥ 85% |
| Real-time processing | ≥ 15 FPS |
| Lane detection/counting | ≥ 85% |